

Lab: Query Database in MongoDB Atlas

Please note that this lab is **not graded**. In this lab exercise, you will learn how to query documents in MongoDB Atlas by creating and refining queries to filter, sort, and paginate movie data. The exercise focuses on the `findMoviesByGenre` function, which retrieves movie titles based on genre. This hands-on activity aims to improve your proficiency in MongoDB querying techniques and enhance the user experience by efficiently managing large datasets. We encourage you to try this lab exercise independently.

Querying Documents:

The most important part of the `findMoviesByGenre` function is the underlying MongoDB query. You will take a step-by-step approach to solving the problem, starting with creating the query on a mongo shell. Once the query has been pre-pared, you will wrap it into a function:

- Create a query to filter results by genre

```
Action genre:
db.movies.find(
  {"genres" : "Action"}
)
```

- The requirement is to **return only the titles** of the movies. For this, add a projection to project only the Title field and exclude the rest, including `_id`

```
db.movies.find(
  {"genres" : "Action"},
  {"_id" : 0, "title" : 1}
)
```

- Now, sort the results in descending order of IMDb ratings. Add a `sort()` function to the query: `db.movies`.

```
db.movies.find(
  {"genres" : "Action"},
  {"_id" : 0, "title" : 1})
.sort({"imdb.rating" : -1})
```

- Add the skip function and, for now, provide any value you want

```
db.movies.find( {"genres" : "Action"},
  {"_id" : 0, "title" : 1})
.sort({"imdb.rating" : -1})
.skip(3)
```

- Next, add a `limit` to the query, as follows. The limit value indicates the page size:

```
db.movies.find(
  {"genres" : "Action"},
  {"_id" : 0, "title" : 1})
.sort({"imdb.rating" : -1})
.skip(3)
.limit(11)
```

- Finally, convert our resulting cursor into an array by using the `toArray()` function:

```
db.movies.find(
  {"genres" : "Action"},
  {"_id" : 0, "title" : 1})
.sort({"imdb.rating" : -1})
.skip(3)
.limit(11)
.toArray()
```

- Now that the query has been written, open a text editor and write an empty function that accepts a genre, a page number, and a page size, as follows:

```
var findMoviesByGenre = function(genre, pageNumber, pageSize){}
```

- Copy and paste the query inside the function, assigning it to a variable, as follows:

```
var findMoviesByGenre = function(genre, pageNumber, pageSize){
  var movies = db.movies.find(
    {"genres" : "Action"},
    {"_id" : 0, "title" :1})
    .sort({"imdb.rating" : -1})
    .skip(3)
    .limit(11)
    .toArray()
  }
}
```

- The result you will get is an array. Write the logic needed to iterate through the elements and print the **title** fields, as follows:

```
var findMoviesByGenre = function(genre, pageNumber, pageSize){
  var movies = db.movies.find(
    {"genres" : "Action"},
    {"_id" : 0, "title" :1})
    .sort({"imdb.rating" : -1})
    .skip(3)
    .limit(11)
    .toArray()
  print("***** Page : " + pageNumber)
  for(var i =0; i < movies.length; i++){
    print(movies[i].title)
  }
}
```

- The query still has hardcoded values that need to be replaced with the variables that are received as function arguments, so put the **genre** and **pageSize** variables in the correct places:

```
var findMoviesByGenre = function(genre, pageNumber, pageSize){
  var movies = db.movies.find(
    {"genres" : genre},
    {"_id" : 0, "title" :1})
    .sort({"imdb.rating" : -1})
    .skip(3)
    .limit(pageSize)
    .toArray()
  print("***** Page : " + pageNumber)
  for(var i =0; i < movies.length; i++){
    print(movies[i].title)
  }
}
```

- Now, you need to derive the skip value based on the page number and pagesize. When the user is on the first page, the skip value should be zero. On thesecond page, the skip value should be the page size. Similarly, if the user is on the third page, the skip value should be page size multiplied by 2. Write thislogic as follows:

```
var findMoviesByGenre = function(genre, pageNumber, pageSize){
  var toSkip = 0;
  if(pageNumber < 2){
    toSkip = 0;
  } else{
    toSkip = (pageNumber -1) * pageSize;
  }
  var movies = db.movies.find(
    {"genres" : genre},
    {"_id" : 0, "title" :1})
    .sort({"imdb.rating" : -1})
    .skip(toSkip)
    .limit(pageSize)
    .toArray()
  print("***** Page : " + pageNumber)
  for(var i =0; i < movies.length; i++){
    print(movies[i].title)
  }
}
```

- Now, use the newly calculated skip value in the limit function. This makes the function complete.
- Copy and paste the function into the mongo shell and execute it. You should see the following result:



```
MongoDB
> var findMoviesByGenre = function(genre, pageNumber, pageSize){
...   var toSkip = 0;
...   if(pageNumber < 2){
...     toSkip = 0;
...   } else{
...     toSkip = (pageNumber -1) * pageSize;
...   }
...   var movies = db.movies.find(
...     {"genres" : genre},
...     {"_id" : 0, "title" :1})
...   .sort({"imdb.rating" : -1})
...   .skip(toSkip)
...   .limit(pageSize)
...   .toArray()
...   print("***** Page : " + pageNumber)
...   for(var i =0; i < movies.length; i++){
...     print(movies[i].title)
...   }
... }
[... ]
> findMoviesByGenre("Action", 5, 3)
***** Page : 5
The Matrix
Battleship Galactica
Driller
>
```

In this activity, by using the **sort()**, **skip()**, and **limit()** functions, you implemented pagination for your movie service, vastly improving the user experience.